

Math Nuggets for GEN AI

Self-Attention Mechanisms: Understanding Sequence Dependencies

By Dr. Anish Roychowdhury, Plaksha University

September 22, 2025

The Challenge: Modeling Long Sequences in Language

Early neural language models faced a fundamental challenge: how to capture dependencies between words that are far apart in a sentence. Consider these examples:

- "The student who studied hard for weeks **passed** the exam."
- "The students who studied hard for weeks **passed** the exam."

The verb "passed" must agree with the subject "student/students," but they're separated by several words. Traditional neural networks struggled with such long-range dependencies, leading to poor performance on complex language tasks.

Word-by-Word Translation: Where Sequential Processing Fails

Consider translating the German sentence "Kannst du mir helfen diesen Satz zu übersetzen" (Can you help me translate this sentence) to English word by word:

Why This Fails:

- **Word Order:** German verb "übersetzen" (translate) comes at the end, but English needs it earlier
- **Context Loss:** Each word is translated independently, losing sentence-level meaning
- **Grammar Mismatch:** Different languages have different syntactic structures
- **Ambiguity:** Words like "Satz" can mean "sentence," "set," or "clause" depending on context

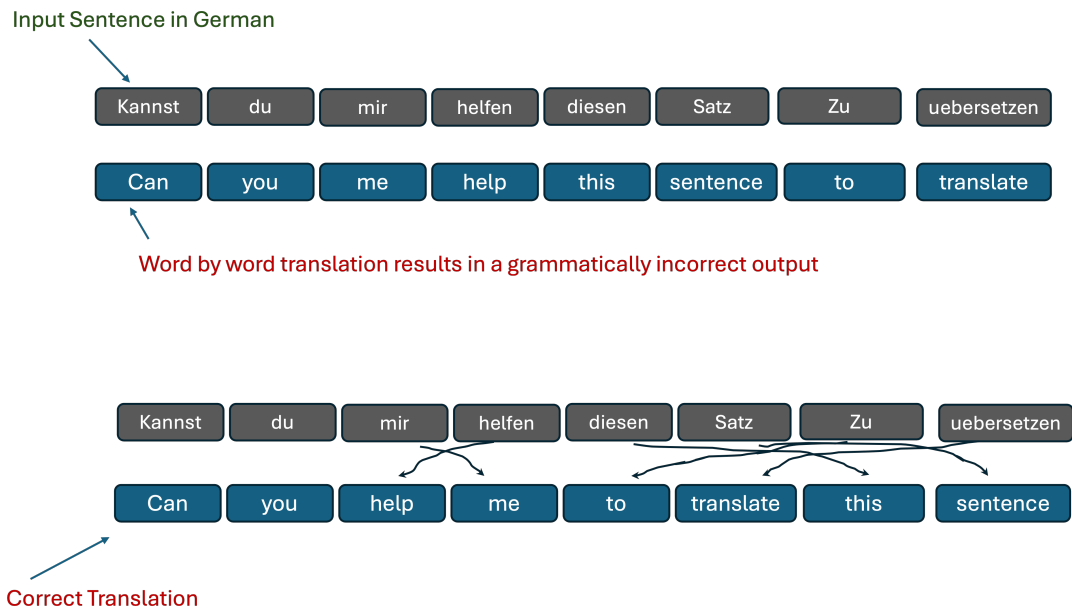


Figure 1: Word-by-word translation fails due to different grammatical structures between languages

The solution requires understanding the *entire* input sequence before generating any output.

Encoder-Decoder Architecture: Processing Complete Sequences

To address word-by-word translation limitations, researchers developed encoder-decoder architectures:

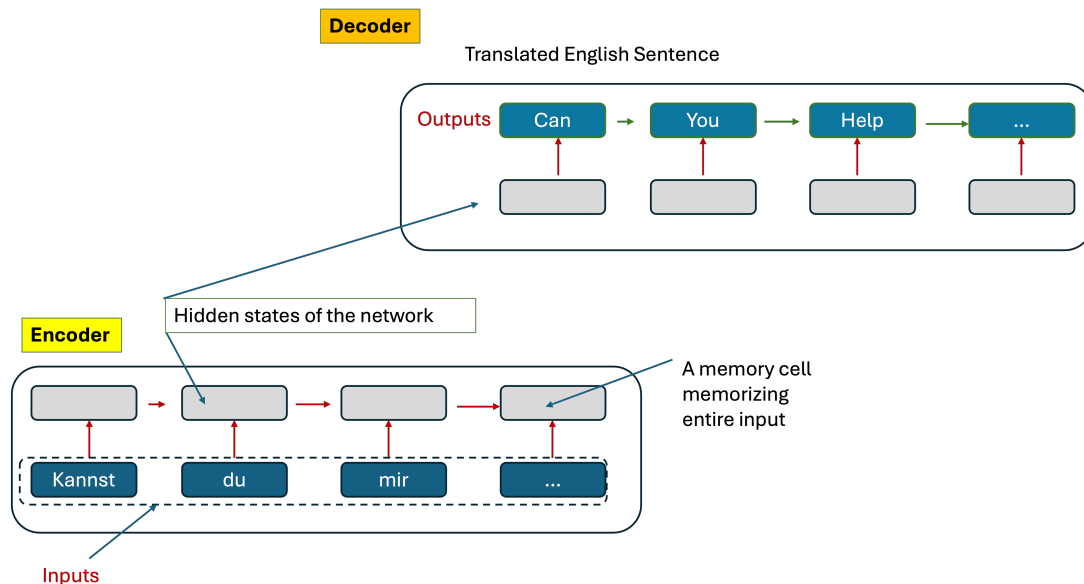


Figure 2: Encoder-decoder architecture processes the entire input sequence before generating output

How It Works:

- **Encoder:** Reads and processes the entire input sequence into a fixed-size representation
- **Hidden State:** Captures the "meaning" of the complete input sequence
- **Decoder:** Uses this representation to generate the output sequence word by word
- **Context Preservation:** Maintains global context throughout translation

This approach was a significant improvement, but it introduced new challenges with very long sequences.

RNN Limitations in Sequence Modeling

Recurrent Neural Networks (RNNs) were the first successful implementation of encoder-decoder architectures, but they had critical limitations:

The Bottleneck Problem:

- **Information Compression:** Entire input sequence compressed into single hidden state vector

- **Gradient Vanishing:** Information from early tokens gets "forgotten" in long sequences
- **Sequential Processing:** Cannot process tokens in parallel, leading to slow training
- **Fixed Context:** Decoder only sees final hidden state, losing access to earlier information

Mathematical Representation: For an RNN, the hidden state h_t at time t is:

$$h_t = f(h_{t-1}, x_t) \quad (1)$$

Where information from $x_1, x_2, \dots, x_{t-1}$ is compressed through the chain $h_1 \rightarrow h_2 \rightarrow \dots \rightarrow h_t$.

For long sequences, early information becomes increasingly diluted, leading to the "forgetting" problem.

Attention Mechanism: Selective Focus on Relevant Information

The attention mechanism, introduced by Bahdanau et al. (2014), revolutionized sequence modeling by allowing models to selectively focus on relevant parts of the input:

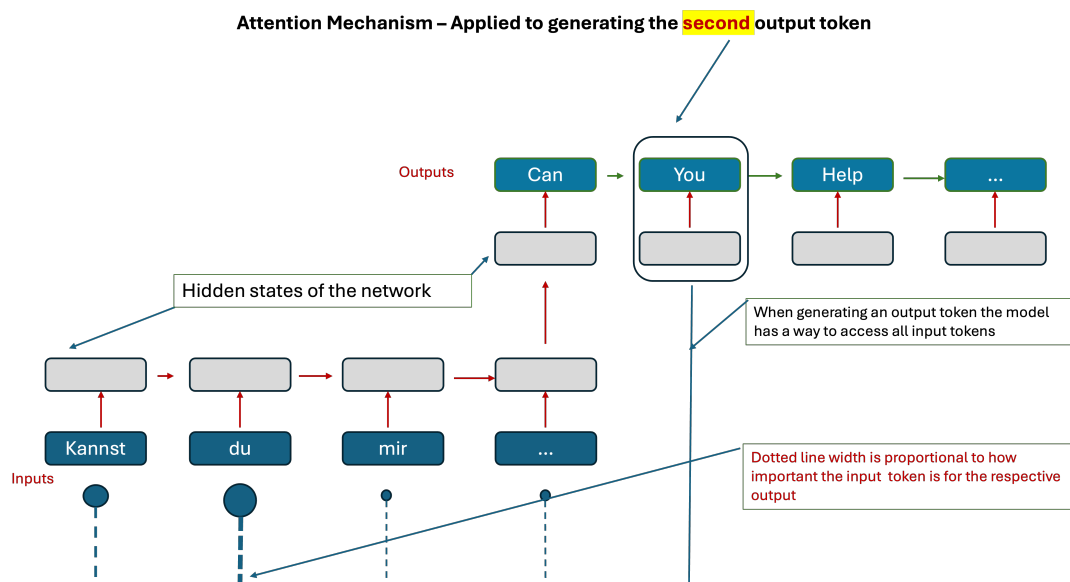


Figure 3: Attention mechanism allows decoder to access all encoder hidden states, not just the final one

Key Innovation: Instead of relying only on the final encoder state, the decoder can *attend* to any part of the input sequence when generating each output token.

Mathematical Foundation: For each output position t , compute attention weights

$\alpha_{t,i}$ for each input position i :

$$e_{t,i} = \text{score}(s_{t-1}, h_i) \quad (\text{similarity score}) \quad (2)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^n \exp(e_{t,j})} \quad (\text{attention weights}) \quad (3)$$

$$c_t = \sum_{i=1}^n \alpha_{t,i} h_i \quad (\text{context vector}) \quad (4)$$

This allows the model to focus on different parts of the input for each output word.

Self-Attention: Attention Within the Same Sequence

While traditional attention connects encoder and decoder states, **self-attention** computes attention weights within the same sequence:

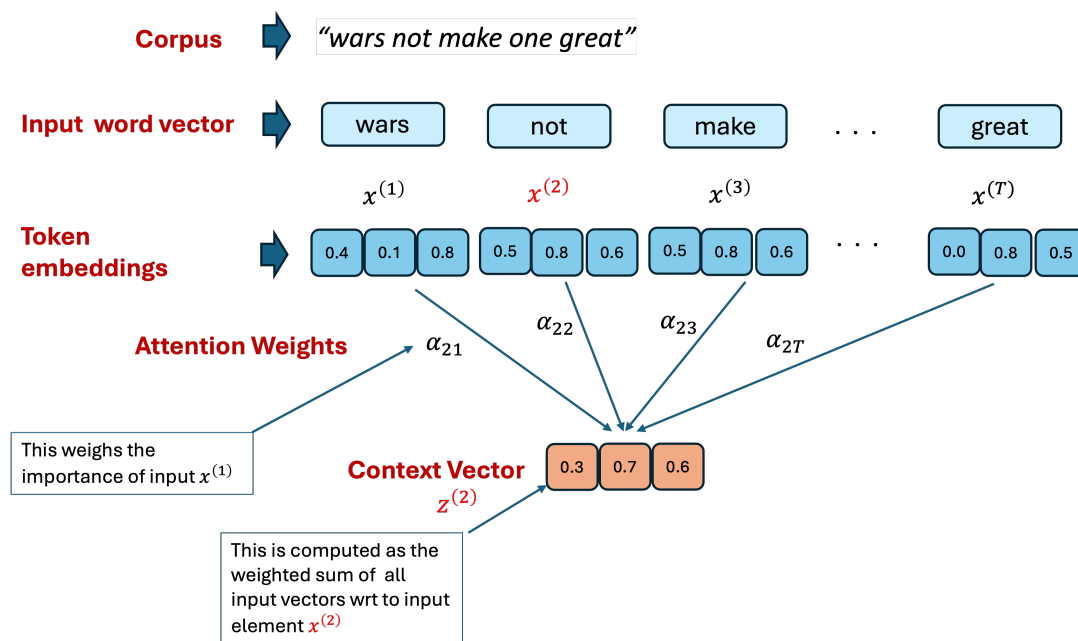


Figure 4: Self-attention allows each position to attend to all positions in the same sequence

Why Self-Attention is Superior:

- **Parallel Processing:** All positions processed simultaneously, not sequentially
- **Long-Range Dependencies:** Direct connections between any two positions in sequence
- **Flexible Context:** Each position gets a customized context vector
- **Normalization:** Softmax ensures attention weights sum to 1, creating probability distributions
- **Interpretability:** Attention weights show which words the model focuses on

Key Difference from Standard Attention:

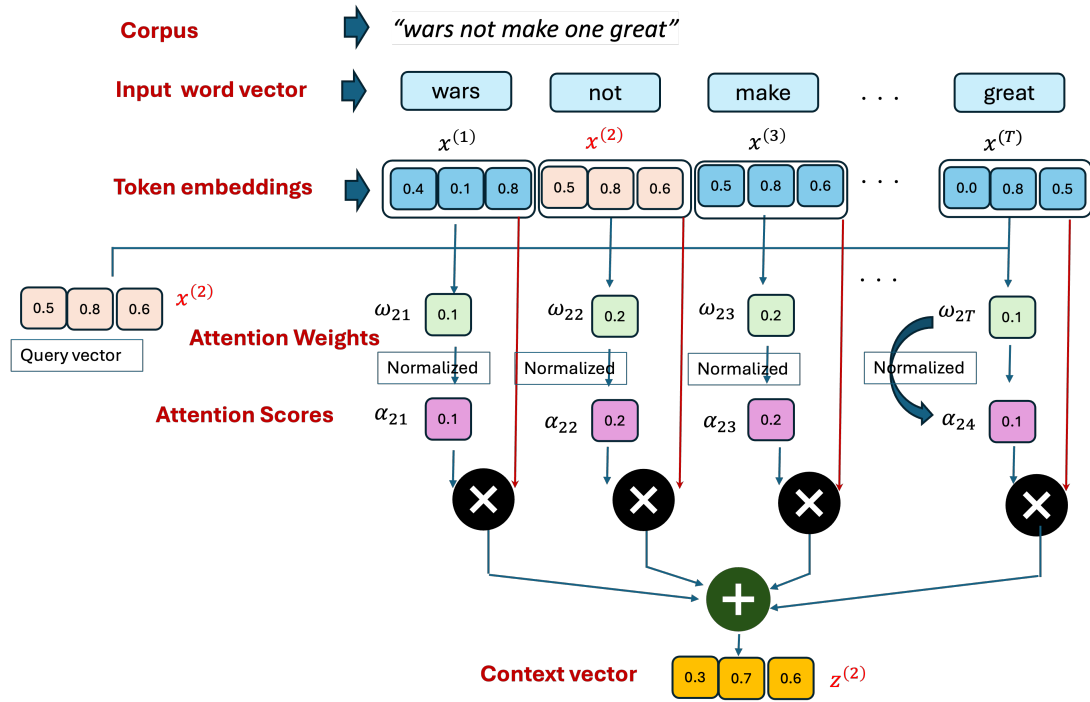


Figure 5: Self-attention with normalization showing how attention weights form probability distributions

- **Standard Attention:** Connects two different sequences (encoder $\rightarrow$ decoder)
- **Self-Attention:** Connects positions within the same sequence

Query-Key-Value Framework: The Mathematical Foundation

Self-attention operates through three learned transformations of the input:

Conceptual Understanding:

- **Query (Q):** "What am I looking for?"
- **Key (K):** "What can I offer as context?"
- **Value (V):** "What information do I actually provide?"

Mathematical Formulation: Given input embeddings $X \in \mathbb{R}^{n \times d}$ where n is sequence length and d is embedding dimension:

$$Q = XW^Q \quad \text{where } W^Q \in \mathbb{R}^{d \times d_k} \quad (5)$$

$$K = XW^K \quad \text{where } W^K \in \mathbb{R}^{d \times d_k} \quad (6)$$

$$V = XW^V \quad \text{where } W^V \in \mathbb{R}^{d \times d_v} \quad (7)$$

Step-by-Step Self-Attention Computation

Let's work through a concrete example with sequence "Alice loves Bob":

Step 1: Create Q, K, V matrices

$$\text{Input: } X = \begin{bmatrix} \mathbf{x}_{\text{Alice}} \\ \mathbf{x}_{\text{loves}} \\ \mathbf{x}_{\text{Bob}} \end{bmatrix} \in \mathbb{R}^{3 \times d} \quad (8)$$

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V \quad (9)$$

Step 2: Compute attention scores

$$\text{Scores} = QK^T = \begin{bmatrix} q_1 \cdot k_1 & q_1 \cdot k_2 & q_1 \cdot k_3 \\ q_2 \cdot k_1 & q_2 \cdot k_2 & q_2 \cdot k_3 \\ q_3 \cdot k_1 & q_3 \cdot k_2 & q_3 \cdot k_3 \end{bmatrix} \quad (10)$$

Step 3: Apply scaled softmax

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (11)$$

The scaling factor $\frac{1}{\sqrt{d_k}}$ prevents softmax from saturating for large embedding dimensions.

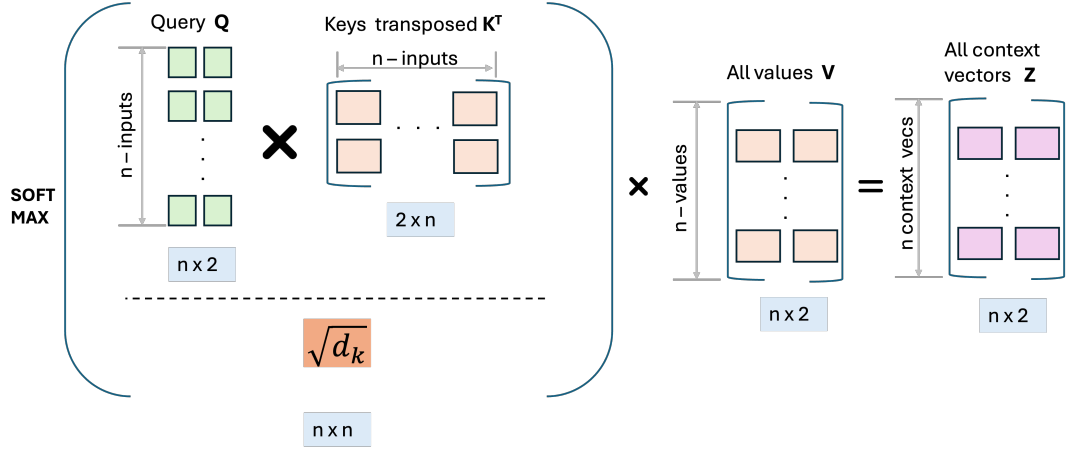


Figure 6: Complete self-attention computation pipeline showing Query-Key-Value transformations

Numerical Example

Let's compute self-attention for a simplified 2D example:

Given:

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \quad (3 \text{ tokens, } 2\text{D embeddings}) \quad (12)$$

$$W^Q = W^K = W^V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \quad (\text{identity for simplicity}) \quad (13)$$

Step 1: $Q = K = V = X$ (since weights are identity)

Step 2: Compute QK^T :

$$QK^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix} \quad (14)$$

Step 3: Apply softmax (with $\sqrt{d_k} = \sqrt{2} \approx 1.41$):

$$\text{Scaled scores} = \frac{1}{1.41} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix} \quad (15)$$

$$\text{Attention weights} = \text{softmax}(\text{scaled scores}) \quad (16)$$

Step 4: Multiply by values V to get final output representations.

Complete Self-Attention Pipeline

The following figure consolidates the entire self-attention process, showing how input embeddings are transformed through the Query-Key-Value mechanism to produce context-aware representations:

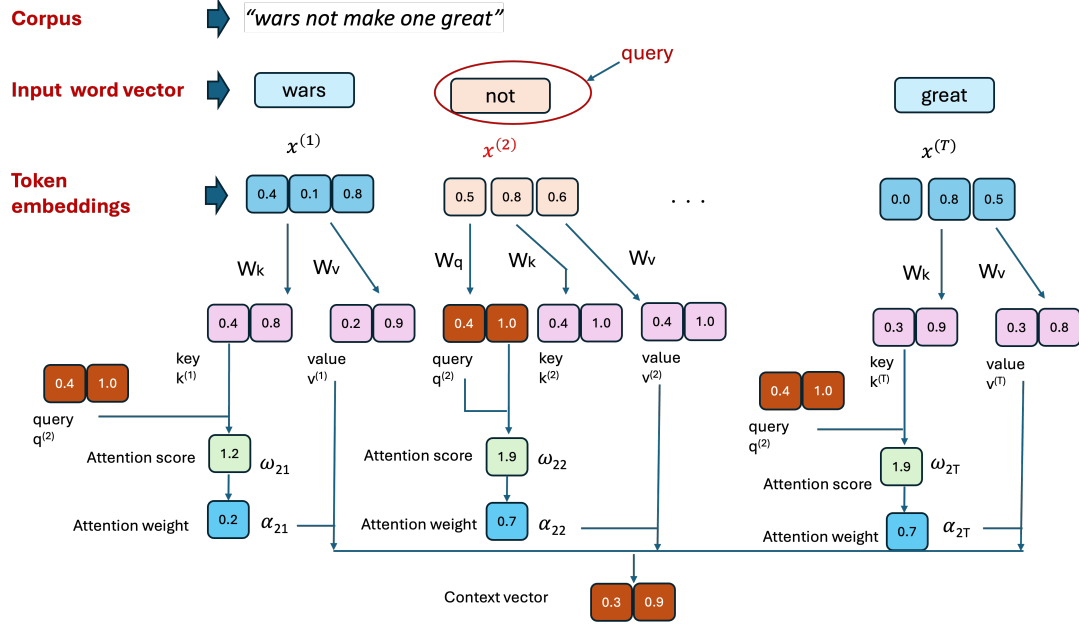


Figure 7: Consolidated view of self-attention computation showing the complete pipeline from input embeddings through Q-K-V transformations, attention score calculation, normalization, and final context vector generation

This consolidated view demonstrates how each token (using "not" as the query example) attends to all positions in the sequence, computing attention scores through dot products of queries and keys, normalizing these scores via softmax to create probability distributions, and finally combining the normalized weights with value vectors to produce enriched representations that capture contextual relationships across the entire sequence.

Key Takeaways

1. **Problem:** RNNs struggle with long-range dependencies due to sequential processing
2. **Solution:** Self-attention enables direct connections between any positions
3. **Mechanism:** Query-Key-Value framework allows flexible, learned attention patterns
4. **Advantage:** Parallel processing and interpretable attention weights

5. **Foundation:** This mechanism forms the core of transformer architectures

Self-attention revolutionized natural language processing by solving the fundamental challenge of capturing long-range dependencies efficiently, paving the way for modern language models.